ANÁLISE COMPLETA

% % % % % % % % % % % % % % % % % %

Sistema Gráfica Máxima

Squad 01 — Residência em Software — UCB / Porto Digital

Análise técnica do código-fonte, arquitetura, banco de dados,
funcionalidades e comparação com a documentação oficial.

Setembro 2026

1. Visão Geral do Projeto

Informações gerais sobre o sistema e a stack tecnológica

Aspecto	Detalhe
Nome	Gráfica Máxima — Squad 01 (MVP)
Contexto	Projeto de Residência em Software — Porto Digital / UCB
Objetivo	Centralizar orçamentos, substituir processos manuais e melhorar o controle financeiro
Stack Frontend	React 19.2 + Vite 8 + React Router 7 + Chart.js + Axios + Lucide React
Stack Backend	Node.js + Express 5 + MySQL2 + PDFKit + JWT + dotenv + CORS
Banco de Dados	MariaDB 10.4 via XAMPP (12 tabelas, relacionamentos FK)
Repositório	github.com/llannkkj/squad01-graficamaxima-mvp

2. Arquitetura do Sistema

O sistema segue uma arquitetura cliente-servidor com separação clara entre Frontend e Backend:

Frontend (React + Vite)

- App.jsx — Componente raiz com BrowserRouter e 7 rotas
- 6 Páginas: Dashboard, Orçamentos, NovoOrcamentos, StatusOrcamento, Config, Papéis
- 6 Componentes reutilizáveis: Menu, Cards, Gráfico, Tabela, StepItens, CalculadoraM2
- 1 Service: api.js (instância Axios apontando para localhost:3301)
- 6 Arquivos CSS modulares: app.css, calculadora.css, config.css, etc.

Backend (Node.js + Express)

- server.js — Entry point com 3 grupos de rotas registrados
- Padrão MVC: routes/ !’ controllers/ !’ services/ !’ config/db
- 4 Controllers: orcamentoController, MetroQuadradoController, AuthController, DashBoardController
- 6 Services: MetroQuadradoService, OrcamentoService, orcamentoPdf.service, LogService, TaxaServico, precos.json
- 1 Middleware: AuthRole.js (JWT — atualmente desativado)

Banco de Dados (MariaDB)

O banco "graficamaxima" possui 12 tabelas com constraints FK:

- Tabelas core: orcamento, cliente, item, usuario
- Tabelas de insumos: papel, impressao, acabamento, corte, servico
- Tabelas associativas: acabamento_item, item_corte, servico_item
- Tabelas auxiliares: documento, historico_log

> **!9 NOTA**
> As tabelas documento, historico_log, servico e servico_item existem no schema mas NÃO são utilizadas pelo código da aplicação.

3. Funcionalidades: Implementação vs. Documentação

Comparação entre o que foi documentado no PDF e o que existe no código

Funcionalidade (Doc PDF)	Status	Observação
Dashboard com métricas	& þ Parcial	Cards com valores hardcoded (R$1200, 12) — não consulta banco
Gráfico de evolução mensal	& þ Parcial	Componente grafico.jsx existe mas com dados estáticos
CRUD de Orçamentos	' Completo	Criar, listar, editar, deletar — conectado ao banco com transações
Fluxo em 3 etapas	' Completo	Modal wizard: Cliente !' Serviço !' Custos de produção
Status/Andamento do orçamento	' Completo	Stepper visual + checkboxes + persistência no banco
Geração de PDF	' Completo	PDFKit no backend, download via blob no frontend
Calculadora de M² (flutuante)	' Completo	Botão FAB global + integração com API de cálculo
Configurações (abas)	' Completo	CRUD de acabamentos/corte por abas + Metro Quadrado via JSON
Gerenciamento de Papéis	& þ Parcial	Página Papeis.jsx existe com funcionalidade limitada
Cadastro/Login de Usuário	'L Ausente	Middleware JWT existe, rotas de auth comentadas no server.js
Tela de Login/Registro	'L Ausente	Documentação mostra tela, não existe no código
Motor de aproveitamento de folha	'L Ausente	Documentado como "diferencial técnico", não implementado
Imutabilidade ACID de preços	'L Ausente	Valores não são congelados no orçamento
Auditoria / Logs	'L Ausente	Tabela historico_log e LogService existem mas não são usados
CI/CD Pipeline	'L Ausente	Documentação menciona mas não há .github/workflows
Testes com Jest	'L Ausente	Documentação menciona, Jest não está nas dependências

4. Análise de Qualidade do Código

Pontos positivos, problemas de segurança e débitos técnicos

4.1 Pontos Positivos

- Transações ACID no criarOrcamento — usa beginTransaction, commit e rollback
- Conexão resiliente ao banco — fallback automático entre portas 3306 e 3312
- Blindagem de FK — verifica/insere registros padrão de papel e impressao antes de criar orçamento
- Fallback na listagem — se JOIN falhar, faz SELECT simples como contingência
- Formatação BR completa — moeda, datas e números com Intl.NumberFormat
- Masks de input — CPF/CNPJ e CEP com formatação automática no frontend
- Geração de PDF bem estruturada — layout profissional com cabeçalho, tabela e rodapé via PDFKit

4.2 Problemas de Segurança

Ø=Ý4 CRÍTICO

Arquivo .env commitado no repositório com credenciais do banco (DB_USER=root, DB_PASS vazia). O .env deveria estar no .gitignore.

Ø=Ý4 CRÍTICO

Autenticação completamente desativada — as rotas de auth estão comentadas no server.js. Qualquer pessoa na rede pode acessar/modificar todos os dados.

Ø=Ý4 CRÍTICO

Senha do usuário admin no SQL dump é "hash_seguro_123" — armazenada em texto plano, sem bcrypt ou similar.

4.3 Débitos Técnicos

Problema	Arquivo	Impacto
Dashboard com dados hardcoded	Dashboard.jsx	Cards mostram R$1200 fixo, não refletem dados reais
node_modules no repositório	README.md menciona	Repositório inflado desnecessariamente (~100MB+)
Rotas duplicadas com/sem /api	configuracoesRoutes.js	Cada rota registrada 2x para contornar bugs
nodemon como dependency	package.json (backend)	Deveria ser devDependency
CORS fixo para localhost:5173	server.js	Não permite deploy em produção
Comentários // no .env	.env	.env usa # para comentários; // é parte do valor
Preços em JSON no filesystem	precos.json	Mistura DB + arquivo; risco de perda em deploy
Race condition no DB init	db.js	setTimeout(500ms) como workaround se query chegar antes do pool
Arquivos vazios no backend	Vários (4+ arquivos)	financeiroRoutes, pedidosRoutes, produtosRoutes, etc.

Uso de alert() para feedback	Múltiplos arquivos	UX ruim — deveria usar toast/snackbar
Inline styles extensivos	App.jsx, NovoOrcamentos	Mistura CSS em arquivo e inline, dificulta manutenção

5. Mapeamento de Endpoints da API

Todos os 15 endpoints REST implementados no backend

Método	Rota	Função
GET	/api/orcamentos	Listar todos os orçamentos
POST	/api/orcamentos	Criar orçamento (transacional)
PUT	/api/orcamentos/:id	Editar orçamento
DELETE	/api/orcamentos/:id	Deletar orçamento
GET	/api/orcamentos/status-orcamento/:id	Obter status de um orçamento
PUT	/api/orcamentos/status-orcamento/:id	Atualizar status
GET	/api/orcamentos/:id/pdf	Gerar PDF do orçamento
GET	/api/calculo/tipos	Listar tipos e preços por m²
POST	/api/calculo/metro-quadrado	Calcular e salvar item m²
POST	/api/calculo/metro-quadrado/simulacao	Simulação (sem salvar)
PUT	/api/calculo/tipos/atualizar	Atualizar preço no JSON
GET	/configuracoes/:aba	Buscar configurações por aba
POST	/configuracoes/:aba	Criar item de configuração
PUT	/configuracoes/:aba/:id	Editar item de configuração
DELETE	/configuracoes/:aba/:id	Remover item de configuração

6. Cobertura de Testes

> **& ATENÇÃO**
> Não há testes automatizados. A documentação menciona Jest e ESLint como "Quality Gate", porém: (1) Não há pasta tests/ ou __tests__; (2) Jest não está nas dependências; (3) ESLint está configurado mas sem regras customizadas; (4) Nenhum script de teste no package.json do backend.

7. Gaps: Documentação vs. Realidade

A documentação PDF (21 páginas) descreve um sistema mais completo do que o implementado. Funcionalidades documentadas mas NÃO implementadas:

- Sistema de Autenticação/Login — Tela de "Crie uma Conta" no PDF, código comentado no server.js
- Motor de aproveitamento de folha — Mencionado como "diferencial técnico" na documentação
- Imutabilidade de preços em orçamentos — Documentada como "ACID", valores não são congelados
- Dashboard dinâmico — PDF mostra R$ 9.700 + gráficos reais, código mostra valores fixos
- Caching em memória — Mencionado como "resiliência" na documentação
- Auditoria/Logs — Tabelas existem mas nenhum log é gravado pelo sistema
- CI/CD Pipeline — Documentação menciona, sem .github/workflows no repositório
- Testes unitários com Jest — Documentação menciona como quality gate

8. Recomendações Prioritárias

Ações organizadas por urgência para evolução do sistema

Ø=Ý4 Críticas (Fazer Agora)

- Remover .env do repositório e adicionar ao .gitignore
- Ativar autenticação — descomentar e implementar rotas de auth com JWT
- Hashear senhas — usar bcrypt no lugar de texto plano no banco
- Remover node_modules do repositório — usar .gitignore corretamente

Ø=ßá Importantes (Próxima Sprint)

- Dinamizar o Dashboard — fazer cards consumirem dados reais via API
- Unificar persistência — migrar precos.json para tabela no banco de dados
- Adicionar testes — pelo menos para cálculos de m² e criação de orçamentos
- Corrigir .env — trocar // por # nos comentários
- Remover rotas duplicadas /configuracoes e /api/configuracoes

Ø=ßâ Melhorias (Futuro)

- Substituir alert() por toasts — usar react-hot-toast ou similar
- Implementar auditoria — ativar LogService e gravar logs de alterações
- Adicionar validação de formulário — campos obrigatórios no frontend
- Configurar CORS dinâmico via variável de ambiente
- Mover nodemon para devDependencies
- Extrair estilos inline para os arquivos CSS existentes

9. Conclusão

O Gráfica Máxima é um MVP funcional que cumpre seu objetivo central: permitir a criação, gestão e acompanhamento de orçamentos gráficos. O fluxo de criação em 3 etapas, o cálculo de m², a geração de PDF e o gerenciamento de configurações estão bem implementados e integrados.

No entanto, há uma divergência significativa entre a documentação e o código. A apresentação descreve funcionalidades (autenticação, motor de folha, imutabilidade ACID, CI/CD, testes) que não existem na implementação atual.

As questões de segurança (credenciais expostas, autenticação desativada, senhas em texto plano) são os pontos mais críticos a serem endereçados imediatamente.

Para um projeto de residência/MVP, o resultado é sólido na parte funcional core, mas precisa de trabalho adicional nas áreas de segurança, testes e alinhamento com a documentação.

SCORECARD RESUMO

Funcionalidade Core	Segurança	Qualidade de Código	Testes	Doc vs. Código
7.5/10	**3/10**	**6/10**	**0/10**	**5/10**